

Need help debugging your code? Try some of these tips and tricks!

Read the traceback (the error message itself)

Error messages can be long, confusing, and full of jargon. Consider learning more about the structure of error messages in order to be able to read them more quickly. Here is some information borrowed from [this article](#).

Error messages normally have the following structure:

```
Traceback for most recent call
Location of the program
Line in the program where error was encountered
Name of the error: relevant information about the exception
```

Here is an example of an error message:

```
Traceback (most recent call last):
  File "C:/Python27/hdg.py", line 5, in
    value = A[5]
IndexError: list index out of range
```

In this case, we can see that there is an issue in line 5 of the file `hdg.py`. Specifically, the “list index is out of range,” meaning that the vector `A` must not have an element corresponding to the index 5.

Oftentimes, error messages will look very long because they delineate the full path of execution for the line producing your error. For instance, if you have a line of code in `func3()` that produces an error, and `func3` is called by `func2`, which is called by `func1`, then your error message will first display `func1`, then `func2`, then `func3`, and then finally the line of code that actually causes the error. When in doubt, scroll down to the bottom of your error message to see the individual line that produces the issue.

Use PDB/Breakpoints

PDB is the Python debugger. You can use it by simply calling the function ``breakpoint()`` wherever you would like to place a breakpoint. When you’re executing your program, it will stop at the breakpoint, at which point you can print out the values of various variables. Use the command ``continue`` to start running the program again (until the next breakpoint) or ``next`` to run strictly the next line of code. Refer to [the PDB documentation](#) for more information.

Print out your variables

Although some consider it taboo, adding print statements to your code can be remarkably effective for debugging. Are you wondering why you see an error related to a certain line of code? Print out the variables in that line!

Google it!

99% of the time, someone else has experienced the exact same error as you. Popular websites such as StackOverflow can provide insights regarding the nature of your issue and how to fix it. For instance, if you have read the error message and don't understand the underlying issue, someone online may have already explained it many years ago.



Use a smaller dataset

Oftentimes, it is helpful to first implement your code on a small dataset rather than the full-length dataset you eventually want to use. This can help you to validate your code against answers that you can calculate by hand.

For machine learning models in particular, it can be very helpful to *overfit* your model to a very small dataset to ensure that your code is actually changing the model's weights in a manner that causes it to actually learn. Only after ensuring your loss function decreases to a very small value on a small dataset should you move on to the full-size dataset.

Understand the problem conceptually

If your bug is a logic error which does not terminate your program but produces undesired output e.g. failing an autograder testcase, take a step back from your code to check your understanding of the problem conceptually. What are the key components and relationships involved? Some students make logic errors when blindly translating instructions from the spec to code. To aid in your conceptual understanding, you might want to refer back to the lecture slides and try solving a mini test case by hand on paper. After solving the problem by hand, compare your manual solution with your code. Do they match up? If not, this is a strong indication there is a logical error in your code. Look for discrepancies between your manual solution and the code, and try to pinpoint where things went wrong.

I've tried these, but my code still doesn't work

That's alright! Feel free to make a private post on Piazza. Just make sure to include the following:

- A description of the issue itself
- The shortest possible snippet of code that you know contains your issue. Please paste the code into a code box (one of the Piazza formatting buttons) rather than posting a screenshot of your code
- A description of the debugging techniques you have already used, along with their outcomes
- A description of the inputs (or outputs) that give rise to your issue. For instance, what are the data types of the variables you're dealing with? What are their values? If you're dealing with vectors/arrays/tensors, what are their shapes?

Credits:

- Thanks to the EECS 487 Fall 2023 staff (Jake Sansom, Gordon Lim, Kaijian Zou) for helping to put together the first version of this document!